# OpenGL and WebGL Pipelines

3

The evolution of OpenGL framework both on desktop computers and mobile devices was clearly boosted by the evolution of the graphics processing engines (GPU)s that keep becoming more powerful every day. In fact, the architectural changes in hardware were introducing more and more computing power that could not be easily exploited by the traditional API. This pushed the progressive evolution from the fixed graphics pipeline in versions 1.X, to the possibility of optionally supplying vertex and fragment shaders in version 2.0, until their mandatory use from version 3.1 on.

## 3.1 From models to the processing stages

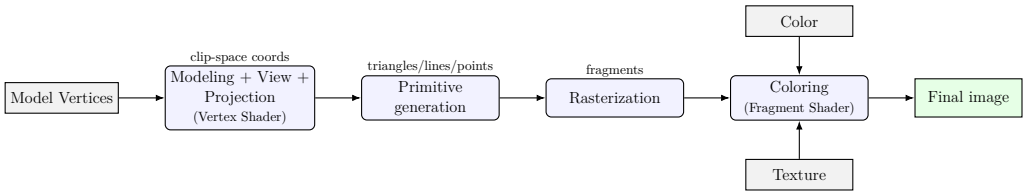



Figure 3.1: Simplified processing pipeline

Despite the different ways of modelling three-dimensional bodies, computer graphics are basically supported by polygonal meshes. This means that any object model will be composed of discrete planar polygons connected by vertices and/or edges. Modern OpenGL and WebGL limits these polygons to triangles, what is not a problem, as any planar polygon can be decomposed into a set of triangles. A triangle is completely defined by three

vertices, and triangles can be grouped for form models that can go from a single face to complex scenes. As models represent objects whose surfaces have distinct colours or patterns, each triangular patch has to be processed as well, in order to produce the final image from a chosen viewpoint.

Computer graphics processing typically follows the order shown in Figure 3.1. It begins by applying geometric transformations to the vertex coordinates to obtain their positions in the output image. After the vertices are grouped into geometric primitives (we are mainly concerned with triangles), another important step assigns a specific colour to each fragment. In modern computer-graphics pipelines, such as the one in Figure 3.2, these two steps must be programmed (with other stages optionally programmable as well). These mandatory stages are called the *vertex shader* and the *fragment shader*.

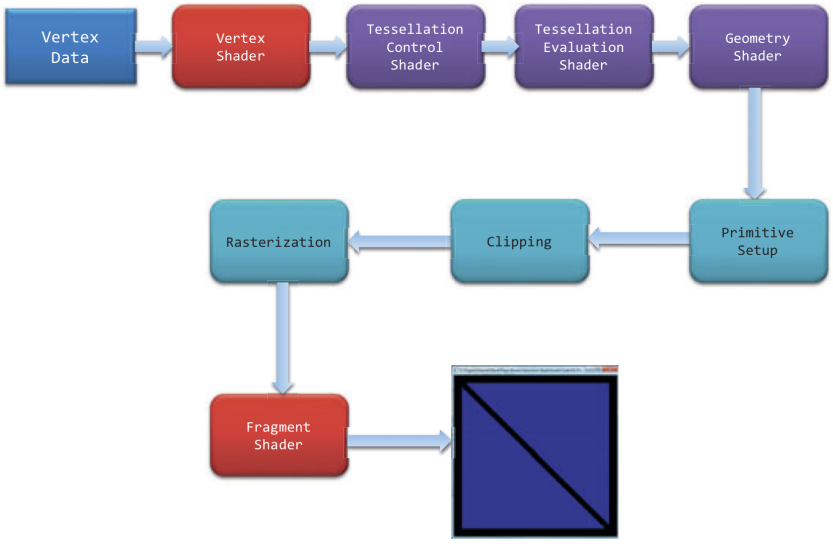



Figure 3.2: The OpenGL 4.6 pipeline

## 3.2 Shaders

As mentioned above, in modern OpenGL pipeline there is place for 5 shaders with different purposes, where the vertex and fragment shaders are mandatory, and the remaining ones optional. As WebGL 1.0 is based on OpenGL ES 1.5[1], so only vertex and fragment shaders can be used.

[1]OpenGL ES X.X refers to the norm of OpenGL that was defined for *embedded* and portable devices

Starting with the vertex shader, we may say that its primary purpose is to apply all the necessary geometric transformations to the vertices that compose each part of the models (or scene) first, and then compute their projection (in the image plane). In reality this projection is not performed as expected from 3D space to a 2D image plane, but we may think of it like that by now.

This shader receives and processes each of the vertices, along with some variables that will influence the transformations applied. For each vertex it computed the position of the corresponding projections in the image plane. This means that the C/C++/Python/JavaScript program will prepare these variables and pass them to the GPU along with the model vertices to be projected.

## 3.3 Vertex Buffer Objects

For the sake of efficiency the model vertices, whether being only three or three million, are passed at once to a structure called Vertex Buffer Object or simply VBO.

Each vertex has 3 coordinates (or $2 + Z = 0$) and may have other attributes, such as colour, texture coordinates or more as needed. Initially these need to be stored in computer main memory and then passed to the graphics processor memory to be processed by graphics pipeline, which includes our programmed shaders. So let's start by create a memory object that will contain the vertices coordinates as follows. Note that the examples will be given primarily in C or C++ and eventually also for Python and Javascript languages.

— **C++** —

```
GLfloat  vertices[6][2] = {
  { -0.90, -0.90 },{  0.85, -0.90 }, { -0.90,  0.85 },// triangle 1
  {  0.90, -0.85 }, {  0.90,  0.90 },{ -0.85,  0.90 } // triangle 2
};
```

— **JavaScript** —

```
this.vertices =
            [ -0.90, -0.90, 0.85, -0.90, -0.90, 0.85, // triangle 1
              0.90, -0.85, 0.90, 0.90, -0.85, 0.90 ]; // triangle 2
```

So let's start with the above sequence of vertices. Note that they were defined in C++ as a bidimensional array of 6 lines by two columns and in JavaScript as unidimensional array of 12 floats. In fact they are equivalent the result and the will be exactly the same, as in fact we will have the 12 floats side-by-side in memory. So for us to use them we need to copy them into the GPU structure called *Vertex Buffer Object* (VBO).

— C++ —

```
GLuint  Buffers[2]; //  one for the vertices and another for the colour
glGenBuffers(2, Buffers);       // In C we may alloc 2 identifications for the requested VBO
glBindBuffer(GL_ARRAY_BUFFER, Buffers[0]); // lets bind the first
// and copy the vertices data
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices),vertices, GL_STATIC_DRAW);

// lets consider the colours are also defined
glBindBuffer(GL_ARRAY_BUFFER, Buffers[1]); // lets bind the second
// and copy the colour data
glBufferData(GL_ARRAY_BUFFER, sizeof(colours),colours, GL_STATIC_DRAW);
```

— JavaScript —

```
this.vertexbuffer=this.gl.createBuffer();
this.gl.bindBuffer(this.gl.ARRAY_BUFFER, this.vertexbuffer);
// as JS stores everything in 64 bit format and GL expects 32bits...
this.gl.bufferData(this.gl.ARRAY_BUFFER, new Float32Array(this.vertices),
                   this.gl.STATIC_DRAW);

this.colorbuffer = this.gl.createBuffer();
this.gl.bindBuffer(this.gl.ARRAY_BUFFER, this.colorbuffer);
// as JS stores everything in 64 bit format and GL expects 32bits...
this.gl.bufferData(this.gl.ARRAY_BUFFER, new Float32Array(this.colors),
                   this.gl.STATIC_DRAW);
```

Note that in the above JavaScript code we use `this.gl.XXX` and in many examples you will see `gl.XXX`. The reason is that `gl` can be seen as an object that we want to keep as a class member that is shared with all its methods.

With the above we have put the data into the GPU memory, but how are the correspondent vertices processed? This is where the vertex shader comes to play.

## The vertex shader

The shaders can be thought of as programs written in a C dialect, that can be put together to form a program. Let's start with the first one that will process our data, the vertex shader.

The vertex shader will basically process the vertices coordinates and manipulates them before passing them to the next step in the pipeline. It may also do colour and other data processing that will be passed down the pipeline to the fragment shader through specially declared variables.

Take a close look at a very simple vertex shader shown hereafter.

— GLSL —

```
attribute  vec2 in_Position;
attribute  vec3 in_Color;

varying  vec3 ex_Color;

```

```
6  void main(void) {
7      gl_Position = vec4(in_Position.x, in_Position.y, 0.0, 1.0);
8
9      // GLSL allows shorthand use of vectors too, the following is also valid:
10     // gl_Position = vec4(in_Position, 0.0, 1.0);
11
12     ex_Color = in_Color;
13 }
```

It starts with the declaration of 2 attributes, `in_Position`, and `in_Color`, which correspond respectively to the vertex coordinates and color. The program also declares an output variable named `ex_Color`, and also uses another one that does not need to be declared because it is a standard predefined variable `gl_Position`.

In this case it is simple to understand that the vertex shader receives a 2D vector from `in_Position` and transforms it into a 4D one to be placed in `gl_Position`. The color here is simply copied to the output variable.

**But how does this processes the 6 vertices passed from the CPU program?** In fact the above program just processes one vertex, but it is automagically called for each of the vertices of the object. Its behaviour does not depend on the type of primitive (line, line loop, triangles, ...) being drawn. It just does its computations on vertices and passes the output variables to be treated at the subsequent levels.

## The vertex shader input attributes

As already said, the vertex shader program is going to be called for each vertex. This means that it must receive all the data corresponding to that vertex, commonly containing its coordinates and colour, but we will see later other to be added.

Each input attribute has to be located from the C/JS program and the linked to the intended VBO. Before drawing becomes possible this has to be done for every input attribute. On the given examples code you can find something like:

— **JavaScript** —

```
1          this.gl.bindBuffer(this.gl.ARRAY_BUFFER, this.vertexbuffer);
2
3          var Location = this.gl.getAttribLocation(this.shaderprog.shaderProgram,
4                                                   "in_Position");
5          this.gl.vertexAttribPointer(Location, // Attribute location
6                                      2, // number of elements per attribute
7                                      this.gl.FLOAT,  // Type of elements
8                                      this.gl.FALSE,
9                                      2*Float32Array.BYTES_PER_ELEMENT,
10                                     0); // Offset from the beginning of the buffer.
```

The above code effectively associates an attribute with a given VBO and its organisation. The same has to be repeated for the colour, and other attributes before calling the drawing primitive.

## The fragment shader

The fragment shader is responsible for computing the final colour of each fragment ("pixel") that becomes visible after projecting the model elements into the output framebuffer. Note that if your scene is composed of multiple objects, each of them will be processed by one shader program, and those shader programs may be the same or totally different from object to object. The same applies naturally to the fragment shader.

Like the vertex shader that is going to be called repeatedly to process one vertex at a time, the fragment shader is going to process individually each fragment. So think of it as "the code that computes the final color for one single fragment" that will be applied to all of them.

— **GLSL** —

```
1 varying  vec3 ex_Color;
2
3 void main(void) {
4     // Pass through our original color with full opacity.
5     gl_FragColor = vec4(ex_Color,1.0);
6 }
```

In the above example it may seem that it receives the exact value passed from the vertex shader `exColor`, but this is not really true. In fact there are (normally) much more fragments than vertices. If the primitives you are drawing are lines, then you have processed the two extremity vertices and passed only two colours to the next stage. As there are a set of fragments between these two extremities, each of them will receive a color (rgb) on this "varying" variable which results from the interpolation of the two extremity colours. So if there is one black (0,0,0) vertex and one white (1,1,1), the fragments in between will receive intermediate grey level values, depending on how close they are to one or another. in the case of triangles, the interpolation will be bilinear between the 3 colours of the 3 vertices.

The output will be written in `gl_FragColor`, another standard variable that effectively consists in the color that will be visible for that fragment in the case it will not be overlapped by another one. Note that the output colour has four components where the fourth is an opacity/transparency factor.

## 3.4 Vertex Array Objects

*You may skip this section this is not available in WebGL 1.0 that we will be using.*

Vertex Array Objects os VAOs were introduced in recent OpenGL versions with the aim of reducing the amount of code needed be repeated every time an object is going to be drawn. You may think of them as an object that will contain the object VBOs and attribute definitions.

— C++ —

```
1  GLuint  VAO[1];
2  GLuint  Buffers[2];
3  const GLuint NumVertices = 6;
4  glGenVertexArrays(1, VAO);
5  glBindVertexArray(VAO[0]);
6  glGenBuffers(2, Buffers);
7  glBindBuffer(GL_ARRAY_BUFFER, Buffers[0]);
8  glBufferData(GL_ARRAY_BUFFER, sizeof(vertices),vertices, GL_STATIC_DRAW);
9  GLuint Location = glGetAttribLocation(shaderprog.shaderProgram,
10                                                          "in_Position");
11 glVertexAttribPointer(Location, 2, GL_FLOAT,
12                                 GL_FALSE, 0, BUFFER_OFFSET(0));
13 glEnableVertexAttribArray(Location);
14
15 glBindBuffer(GL_ARRAY_BUFFER, Buffers[1]); // lets bind the second
16 glBufferData(GL_ARRAY_BUFFER, sizeof(colours),colours, GL_STATIC_DRAW);
17
18 GLuint Location = glGetAttribLocation(shaderprog.shaderProgram,
19                                                          "in_Color");
20 glVertexAttribPointer(Location, 3, GL_FLOAT,
21                                 GL_FALSE, 0, BUFFER_OFFSET(0));
22 glEnableVertexAttribArray(Location);
23 glBindVertexArray(0); // unbind
```

Now considering that we have a data structure (a class) that manages the shaders similar to the used in JavaScript, we may just bind to the VAO and call the draw function.

— C++ —

```
1   glClear(GL_COLOR_BUFFER_BIT);
2   myprog->startUsing();
3   glBindVertexArray(VAOs[0]);
4   glDrawArrays(GL_TRIANGLES, 0, NumVertices);
5   myprog->stopUsing();
```

### Exercises

#### Exercise 3.1

Take the triangle example, and modify the vertex shader so that it will scale the coordinates by 0.5. See the results and try for other scales.

Note that with GLSL you may multiply a vector by a scalar, or simply multiply each of its components.

**Exercise 3.2**
Now modify the vertex shader so that the red component of the colour is always one.

**Exercise 3.3**
Modify the fragment shader so that the blue component will be 0 if its input value is below 0.5, and 1 if not. Do not change the red and green components.

## 3.5 Uniform parameters

As we have seen before, the vertex shader receives the data for a single vertex for performing its computations. So it may be seen as having a different program (or thread) for each vertex. However it is common to have the processing of the whole set of vertices controlled by some parameter that may change during run time. This can be one or more scalars, vectors. and/or matrices. [2]

For this a different type of data exists which are called **uniforms** and consists of one input parameter that is shared between (and common to) the whole set of vertices (or fragments) when processed.

For this we just need to declare the uniform variable on the vertex shader

— **GLSL** —

```
1 uniform float timepar;
```

and then locate it with

— **C++** —

```
1 // in C++
2  GLuint timeLoc   = glGetUniformLocation(myprog->shaderprog, "timepar");
```

or

— **JavaScript** —

```
1 // in JS
2  timeLoc   = gl.getUniformLocation(this.shaderprog.shaderProgram, "timepar");
```

and finally on the display function

[2] Although there was the possibility of adding this parameter repeated for each vertex, this would represent unnecessary overloading both in preparing the data and on the communication between CPU and GPU.

— C++ —

```
float timevariable = somefunctionoftime();
glUniform1f(timeLoc, timevariable);
```

or

For the Javascript case we must be aware that dates are expressed in miliseconds counted from January 1, 1970 00:00:00 UTC. This is indeed a quite big number that, at the time of writing this document was 166487223178. To avoid surprises with the conversion, my suggestion is that you use the time difference from the moment that you started your program.

— JavaScript —

```
var starttime = Date.now();

// later you can compute the difference, but do not forget that
// Date.now() returns and integer value that needs to be passed to a float variable.
var timevariable =(Date.now()-starttime)*1.0;
this.gl.uniform1f(timeLoc, timevariable);
```

## Exercises

### Exercise 3.4

Modify the vertex shader so that the y coordinates of the vertices are added to the sinus of some factor of time, e.g.

$$y_{out} = y_{in} + K_1 \sin(timepar/K_2).$$

Choose $K_1$ and $K_2$ appropriately.

### Exercise 3.5

Create a new version of the shader so that it varies the scale of the object displayed, according to

$$scale = 0.5 + 0.25 \sin(timepar/K2).$$

### Exercise 3.6

Create a new version of the shader that will create the proper rotation matrix, whose angle is related to `timepar` so that it rotates constantly the triangle about the screen centre.

### Exercise 3.7

Modify the fragment shader so that it will vary the green components of the fragments along time, by making them equal to $0.5 + 0.5 \sin(timepar/K2)$.